

Taller de Proyecto II

Informe Final - J2

Camera Tracker

Grupo de Desarrollo

- Deluca, Tadeo - 02655/1
- La Porta, Franco - 02307/2
- Reverte Adriel - 02614/1

Índice

1. Descripción General del Proyecto.....	3
1.1. Introducción.....	3
1.2. Objetivos del Proyecto.....	3
2. Presentación Esquemática del Proyecto.....	4
3. Estructura del proyecto CameraTracker.....	6
3.1. Estructura de archivos.....	6
3.2. Descripción de los archivos principales.....	6
3.3. Configuración de software y dependencias del proyecto.....	8
3.3.1. Configuración de manejadores de placa en el Arduino IDE.....	8
3.3.2. Configuración de la Placa.....	9
3.3.3. Bibliotecas utilizadas en el proyecto.....	9
4. Documentación Relacionada.....	12
4.1. Imágenes.....	12
4.2 Vídeos.....	13
4.3 Enlaces Relevantes.....	13
Apéndice A. Materiales y Presupuesto.....	13
1. Materiales Provistos por la Cátedra.....	13
2. Materiales Provistos por los Alumnos.....	14
3. Evaluación de Presupuesto y Materiales.....	14

1. Descripción General del Proyecto

1.1. Introducción

El proyecto Camera Tracker se desarrolló con el objetivo de implementar un sistema de seguimiento automático de objetos mediante un ESP32-Cam y una torreta con movimiento en dos ejes. Durante las etapas iniciales y medias de desarrollo, se abordaron aspectos clave de hardware, firmware e inteligencia artificial para lograr los objetivos establecidos.

En el transcurso del desarrollo, se enfrentaron desafíos técnicos, como problemas eléctricos y fallas en componentes de hardware, que requirieron soluciones iterativas. Estos ajustes permitieron entregar un sistema funcional capaz de realizar el seguimiento de objetos para el cual fue entrenado, cumpliendo los objetivos primarios propuestos originalmente.

1.2. Objetivos del Proyecto

Objetivos Iniciales

El proyecto fue diseñado para cumplir los siguientes objetivos principales, los cuales fueron cumplidos::

1. **Captura y transmisión de video:** Implementar un sistema de streaming en vivo desde la cámara a través de una interfaz web.
2. **Detección y seguimiento de objetos:** Utilizar un modelo de inteligencia artificial para detectar y rastrear objetos.
3. **Control de la torreta:** Ajustar dos servomotores para alinear la cámara con el objeto detectado, ofreciendo movimiento en dos ejes.
4. **Interacción desde la interfaz web:** Permitir la selección del objeto a rastrear entre dos opciones disponibles (una estatuilla de Super Mario y un mouse azul) y activar/desactivar el streaming de video.

Cambios y Modificaciones

1. Modificaciones sugeridas por la cátedra:

- **Fuente de alimentación externa:** Debido a problemas de consumo eléctrico al conectar el ESP32-Cam y los dos servos, se utilizó una fuente de alimentación externa proporcionada por la cátedra. Esto solucionó los problemas de insuficiencia energética y permitió un funcionamiento estable.

2. Cambios derivados de decisiones de desarrollo:

- **Reemplazo de hardware defectuoso:** Se identificaron fallas en los servomotores **SG90** y en el flex de la cámara del ESP32-Cam. Estas fallas llevaron a la sustitución de componentes dañados, incluyendo la compra de una nueva ESP32-Cam.

3. Modificaciones técnicas propias:

- **Botones para selección de objetos:** Se agregó a la interfaz web la funcionalidad para permitir al usuario seleccionar entre dos objetos predefinidos (Super Mario y el Mouse) como objetivo del seguimiento.
- **Reestilizado de la interfaz web:** Se realizaron mejoras visuales en la interfaz web.
- **Dibujado de la bounding box:** Se implementó una funcionalidad que dibuja un marcador visual (bounding box) alrededor del objeto detectado en la transmisión de video.
- **Optimización de la lógica de movimiento de servos:**
 - **Cambio en la lógica de movimiento:** La lógica inicial, basada en un mapeo directo entre las coordenadas de píxeles en la imagen y los ángulos de los servos, fue reemplazada. En su lugar, se adoptó un enfoque de movimiento en pasos de ángulos discretos hacia la dirección del objeto detectado. Este cambio permitió que el movimiento fuera más suave y controlado, lo que redujo vibraciones y aumentó la precisión aparente del seguimiento.

Objetivos no Cumplidos

Los objetivos secundarios relacionados con la implementación de conexión a servicios en la nube y medidas avanzadas de seguridad (autenticación mediante formulario y protocolo HTTPS) no pudieron ser completados debido a limitaciones de tiempo y retrasos asociadas al hardware del proyecto.

2. Presentación Esquemática del Proyecto

En esta sección presentaremos una actualización completa del proyecto tal como se entrega.

El siguiente gráfico ilustra el flujo desde la captura de una fotografía hasta el procesamiento e inferencia de los datos obtenidos.

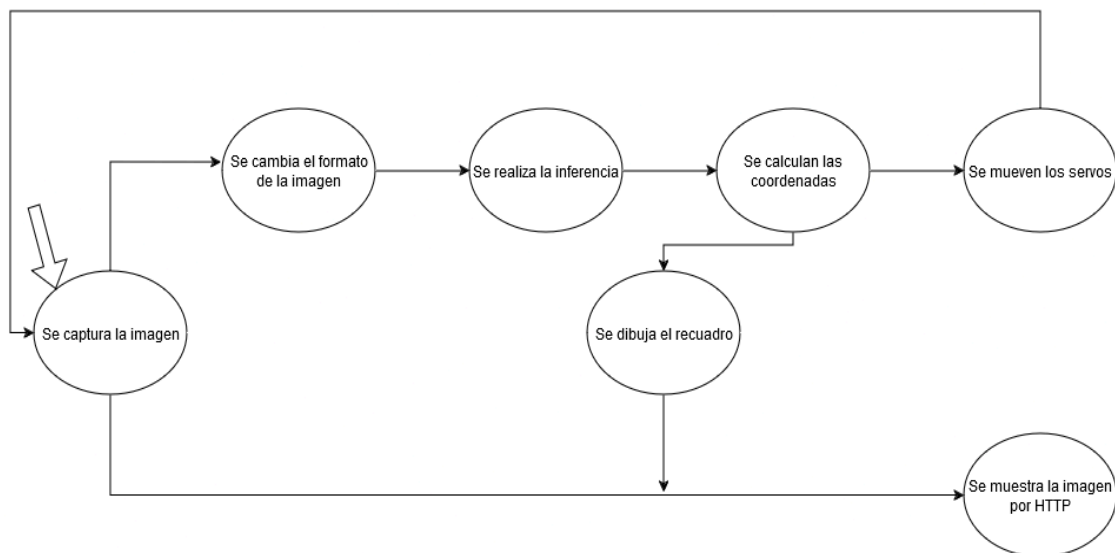


Figura 1. Diagrama de flujo captura de imagen

Se observa cómo la imagen capturada pasa por un cambio de formato para ser utilizada en la inferencia. Luego, si se identifica un objeto seleccionado con una confianza superior al 85%, se realiza la identificación, se marca su posición en la imagen con un recuadro, y se envían señales a los servomotores para ajustar la dirección de la cámara hacia el objeto. Finalmente, la última imagen procesada se transmite en formato JPG al servidor HTTP.

A continuación, se presenta el esquema gráfico que describe la versión estática del sistema desarrollado.

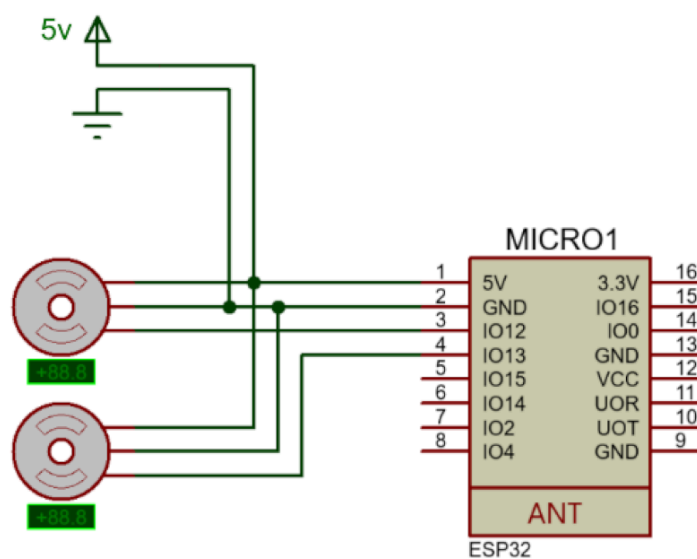


Figura 2. Esquema gráfico físico

En este diagrama, se identifica al ESP32-CAM como el componente principal del sistema, programado mediante un programador USB y alimentado por una fuente de computadora a través de un módulo step-down, que asegura el suministro de energía adecuado. Además, se muestra cómo el ESP32-CAM controla dos servomotores SG90, responsables de permitir el movimiento de la cámara para la función de seguimiento de objetos. Cada módulo y conexión se detalla de manera explícita para facilitar la comprensión del diseño y su implementación.

3. Estructura del proyecto CameraTracker

El proyecto se organiza en una estructura jerárquica que incluye los archivos principales y las librerías necesarias para implementar las funcionalidades del sistema. A continuación, se describe la estructura y el contenido de cada componente.

3.1. Estructura de archivos

```
CameraTracker/  
├─ CameraTracker.ino  
├─ camera_config.h  
├─ servo_control.h  
├─ web_server.h  
└─ libraries/  
    ├─ ESP32Servo  
    └─ lapor-project-1_inferencing
```

3.2. Descripción de los archivos principales

1. Archivo camera_config.h

Este archivo define las configuraciones del módulo de cámara **AI-Thinker ESP32-CAM**, incluyendo los pines GPIO y los parámetros constantes para la captura de imágenes.

Contenido principal:

- Definición de pines GPIO utilizados por la cámara.
- Configuración de las dimensiones del búfer de imágenes capturadas.
- Inclusión de librerías esenciales como esp_camera.h y fb_gfx.h para la configuración de la cámara y manipulación de imágenes..

2. Archivo servo_control.h

Define los pines GPIO asociados con los servos utilizados para el control del movimiento de la cámara y emplea la librería **ESP32Servo** para gestionar su funcionamiento.

3. Archivo web_server.h

Gestiona la configuración y las funcionalidades del servidor web.

Contenido principal:

- Inclusión de librerías esenciales como WiFi.h y esp_http_server.h.
- Configuración de constantes para el manejo del flujo de transmisión, como PART_BOUNDARY.

4. Archivo CameraTracker.ino

Integra la lógica central del proyecto, incluyendo la inicialización de los componentes, la ejecución de las funciones principales y la interacción entre las diferentes partes del sistema.

Funciones principales:

- **setup():**
 - Inicializa la cámara, la conexión Wi-Fi, los servos y las tareas para el streaming y la inferencia.
 - Configura el servidor web.
- **loop():**
 - Vacío debido a que las operaciones continuas son gestionadas por las tareas paralelas inferenceLoop y streamLoop.
- **inferenceLoop():**
 - Ejecuta las inferencias sobre las imágenes capturadas.
 - Ajusta la posición de los servos según los resultados de la inferencia.
- **streamLoop():**
 - Administra el estado del servidor web y lo reinicia si es necesario.

Otras funciones relevantes

1. Inicialización y control de la cámara

- **ei_camera_init():** Configura y activa la cámara.
- **ei_camera_deinit():** Libera los recursos asociados a la cámara.

- **ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf):** Captura una imagen con las dimensiones especificadas por el modelo que realiza la inferencia y la almacena en el búfer proporcionado.

2. Configuración del servidor web

- **startCameraServer():** Configura y lanza el servidor web para la transmisión de imágenes.
- **index_handler(httpd_req_t *req):** Maneja las solicitudes a la página de inicio (/) y proporciona una respuesta HTML.
- **select_handler(httpd_req_t *req):** Procesa solicitudes HTTP relacionadas con la selección de objetos para rastrear.
- **stream_handler(httpd_req_t *req):** Maneja el flujo continuo de video (streaming).

3. Tareas concurrentes

- **inferenceLoop(void *parameter):**
 - Ejecuta inferencias de clasificación sobre imágenes capturadas usando un modelo de **Edge Impulse**.
 - Ajusta la posición de los servos con base en los resultados de la inferencia.
- **streamLoop(void *parameter):** Supervisa el servidor web y lo reinicia si deja de funcionar.

3.3. Configuración de software y dependencias del proyecto

3.3.1. Configuración de manejadores de placa en el Arduino IDE

Manejadores de Placa Instalados

Para el desarrollo del proyecto, se instalaron los siguientes manejadores de placa a través del Board Manager del IDE de Arduino:

- **Arduino AVR Boards:** Versión 1.8.6, utilizada para gestionar placas basadas en microcontroladores AVR en proyectos generales.
- **ESP32 by Espressif Systems:** Versión 2.0.5. Esta versión fue seleccionada por su compatibilidad con las bibliotecas utilizadas en el proyecto, como la biblioteca de inferencia generada por Edge Impulse y la biblioteca WiFi.h. No se utilizó la versión más reciente debido a problemas de compatibilidad.

3.3.2. Configuración de la Placa

La generación y carga del binario se realizó configurando la placa como "**AI-Thinker ESP32-CAM**", correspondiente al módulo de hardware utilizado en el proyecto. Esta configuración fue seleccionada desde el menú de puertos del IDE, asegurando que los parámetros de hardware coincidan con los requisitos del módulo.

3.3.3. Bibliotecas utilizadas en el proyecto

Bibliotecas instaladas mediante el Library Manager

El proyecto requiere la instalación de bibliotecas adicionales que no están incluidas de forma nativa en el IDE de Arduino. Estas bibliotecas deben instalarse a través del **Library Manager**..

- **ESP32Servo**: Versión 3.0.5, desarrollada por Kevin Harrington. Esta biblioteca permite el control de servomotores utilizando señales PWM generadas por el ESP32.

Bibliotecas nativas utilizadas

El proyecto también emplea bibliotecas que ya están incluidas de manera predeterminada en el IDE de Arduino o que no requieren instalación adicional. Estas incluyen:

FreeRTOS

```
#include <freertos/FreeRTOS.h>
```

```
#include <freertos/semphr.h>
```

- Utilizada para la concurrencia y la gestión de tareas en el ESP32, permitiendo implementar semáforos para la sincronización entre tareas.

WiFi y esp_http_server:

```
#include <WiFi.h>
```

```
#include "esp_http_server.h"
```

- Estas bibliotecas son esenciales para la configuración y operación del servidor web.

Arduino Core:

```
#include "Arduino.h"
```

- Incluye funcionalidades básicas y esenciales para la programación y ejecución en plataformas Arduino.

Bibliotecas externas requeridas

Además de las bibliotecas mencionadas, se utilizaron otras librerías específicas que deben ser incluidas manualmente en el proyecto:

Biblioteca de Inferencia Generada por Edge Impulse:

```
#include <lapor-project-1_inferencing.h>
#include "edge-impulse-sdk/dsp/image/image.hpp"
```

- Esta biblioteca es generada al exportar el modelo desde Edge Impulse en formato compatible con el IDE de Arduino. Se importa como archivo comprimido .zip y debe ser incluida en el proyecto a través de la opción "Añadir biblioteca .ZIP" en el IDE. Permite realizar inferencias utilizando el modelo previamente entrenado.

Bibliotecas para Dibujo de la Bounding Box:

```
#include "img_converters.h"
#include "fb_gfx.h"
```

- Estas bibliotecas son necesarias para procesar y renderizar elementos gráficos, como la bounding box, sobre las imágenes capturadas.

3.4. Herramientas de desarrollo y control de versiones

El proyecto utilizó **Git** para control de versiones y **GitHub** como repositorio online, facilitando la colaboración y el seguimiento de cambios. El desarrollo se realizó en el **Arduino IDE**.

3.5. Caso de uso del proyecto

En el siguiente diagrama de secuencia se presenta el caso completo que ocurre cuando un usuario selecciona uno de los objetos disponibles para ser identificado. El diagrama detalla todas las etapas del proceso, comenzando desde la inicialización del servidor web, encargado

de gestionar las interacciones con el sistema, hasta la creación del recuadro que destaca el objeto identificado en la imagen procesada. Además, se incluye el control y ajuste de los servomotores, que permiten al sistema ajustar dinámicamente la posición de la cámara.

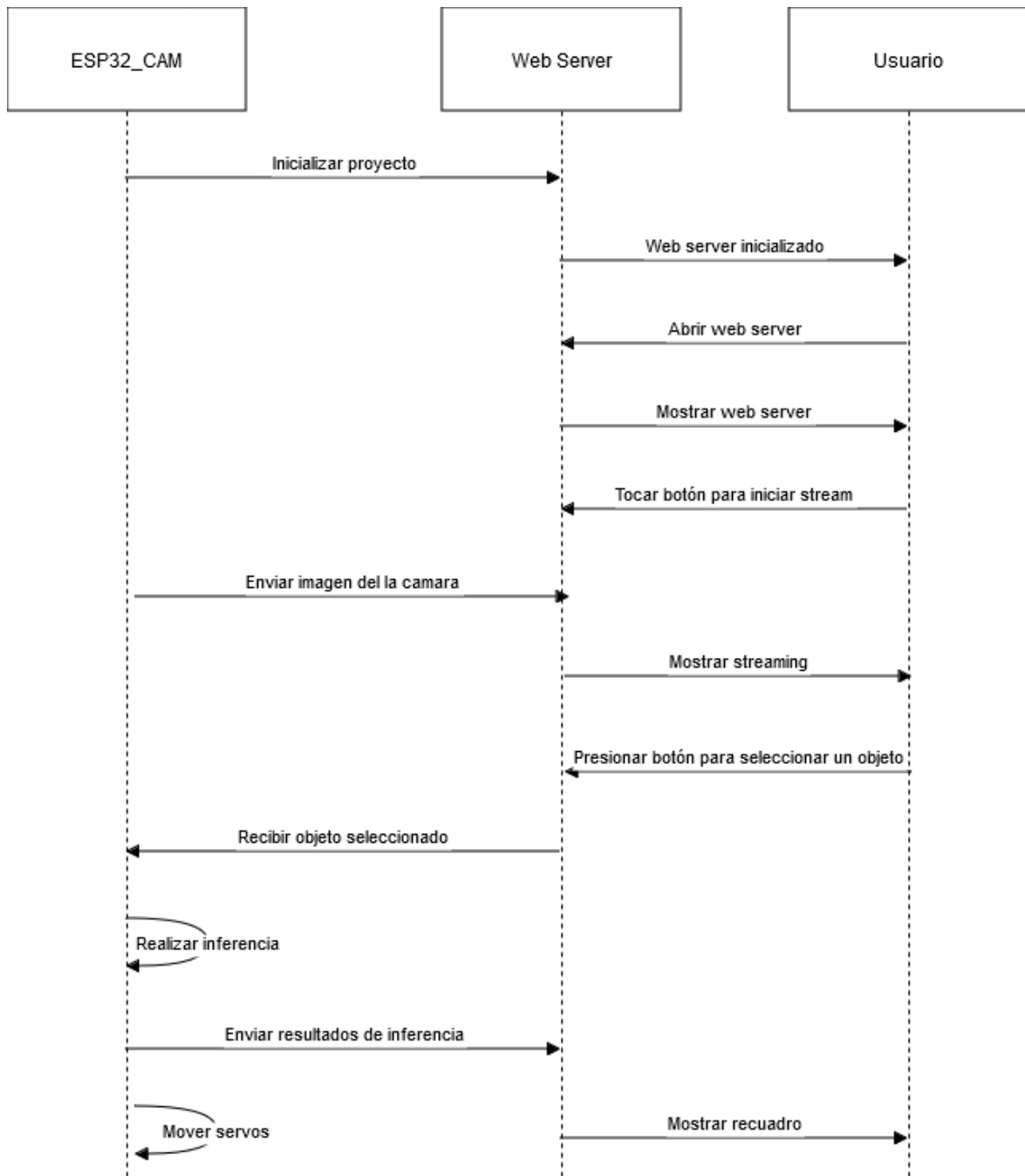


Figura 3. Diagrama de secuencia selección de objetos

4. Documentación Relacionada

4.1. Imágenes

En la primera imagen se muestra la interfaz web desarrollada para el proyecto, destacando las funcionalidades implementadas.

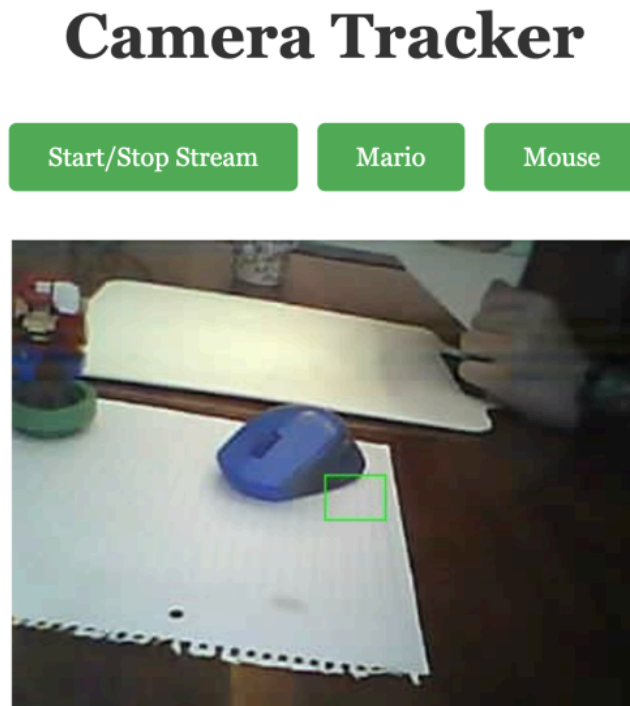


Figura 4. Imagen Web Server

En las siguientes imágenes se puede apreciar el aspecto físico del sistema, incluyendo la estructura completa y los servomotores encargados de controlar el movimiento de la cámara.

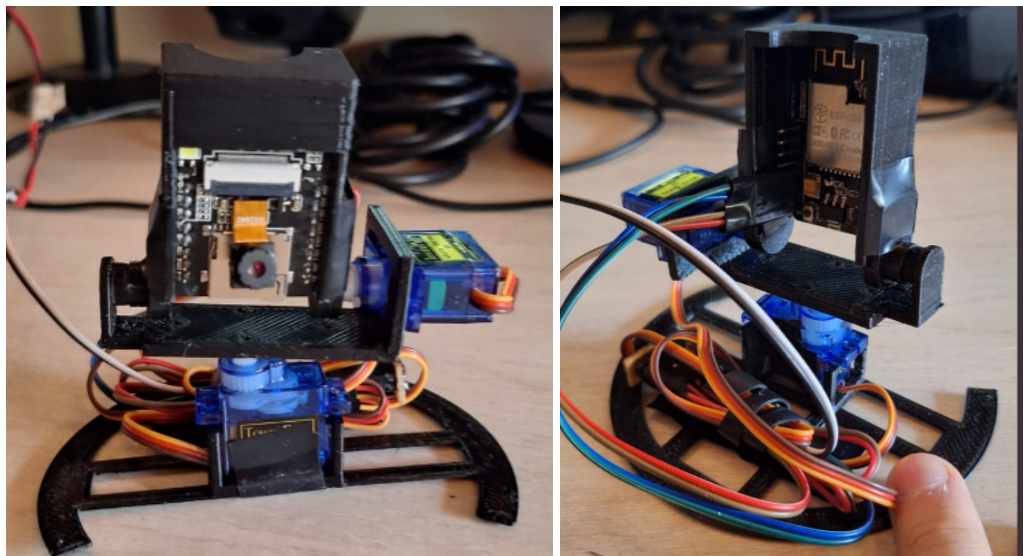


Figura 5. Imagen estructura con servos

4.2 Vídeos

Los vídeos relacionados con el proyecto están disponibles en la carpeta compartida, accesible a través del siguiente enlace: [Carpeta de vídeos del proyecto](#)

En el video titulado "*Camera Tracker video de muestra*", se presenta el proyecto en su totalidad, mostrando de manera clara y detallada su funcionamiento completo.

4.3 Enlaces Relevantes

- **Bitácora del Proyecto:** La bitácora de avances de cada integrante del grupo se encuentra actualizada hasta la fecha y está disponible en el siguiente enlace: [Bitácora](#)
- **Código Fuente:** El código fuente de todos los ejemplos desarrollados hasta el momento está disponible en GitHub: [Repositorio en GitHub](#)
- **Carpeta Compartida del Proyecto:** Todos los archivos de interés relacionados con el proyecto se encuentran en la carpeta compartida accesible a través del siguiente enlace: [Carpeta del Proyecto](#)

Apéndice A. Materiales y Presupuesto

En esta sección se detalla la lista completa y actualizada de los elementos de hardware necesarios para la construcción del proyecto Camera Tracker, diferenciando entre aquellos que fueron provistos por la cátedra y los que fueron adquiridos por los alumnos. Además, se incluyen los costos aproximados de cada uno de estos materiales, basados en precios obtenidos durante el período de desarrollo del proyecto. Cabe aclarar que no se consideran dentro de este presupuesto dispositivos adicionales como computadoras o teléfonos móviles.

1. Materiales Provistos por la Cátedra

La cátedra ha proporcionado varios componentes esenciales para la realización del proyecto, los cuales se detallan a continuación:

- **ESP32-Cam + Programador:** \$19.980,90
[Esp32 Cam Ov2640 + Placa Base Programador Esp32-cam-mb Hobb | MercadoLibre](#)
- **Servo SG90 (x2):** \$2.519 (c/u)
[Mini Servo Tower Pro Sg90 9g Robotica Arduino Servomotor | MercadoLibre](#)
- **Fuente de PC:** \$19.500,00
[Lote 2 Fuentes Atx 24p+p4 400w 450w Usadas Garantia Congreso | MercadoLibre](#)

Estos materiales fueron claves para la implementación inicial del sistema de seguimiento, especialmente para el control de la cámara y la alimentación de la misma.

2. Materiales Provistos por los Alumnos

A continuación, se detallan los materiales adquiridos directamente por los alumnos para complementar el proyecto, los cuales también fueron necesarios para el montaje y pruebas del sistema:

- **Protoboard:** \$4.000,00
[Placa Experimental Protoboard 830 Puntos Arduino Electronica | MercadoLibre](#)
- **Cables Arduino:**
[Cables Para Protoboard Macho Macho 20 Cm X 40 Dupont Arduino | MercadoLibre](#)
- **Filamento PLA:** \$12.999,00
[Filamento Pla Impresora 3d * G S T * 1,75mm 320mt X1 Kg Gst | MercadoLibre](#)
- **Cable USB Micro:** \$2.798,00
[Cable Usb Somostec Micro Trenzado Carga Rápida 3.1 A 1m F | MercadoLibre](#)

3. Evaluación de Presupuesto y Materiales

En general, la mayoría de los materiales esenciales para la construcción y prueba del Camera Tracker fueron provistos por la cátedra, con excepción de algunos componentes auxiliares que fueron adquiridos por los alumnos. Aunque algunos materiales ya se encuentran disponibles, las pruebas y desarrollo del proyecto dependerá de la correcta implementación de estos y otros materiales que puedan ser necesarios a futuro.

La suma total de los componentes adquiridos hasta el momento asciende a:

- **Materiales provistos por la cátedra:** \$44.520
- **Materiales adquiridos por los alumnos:** \$19.800